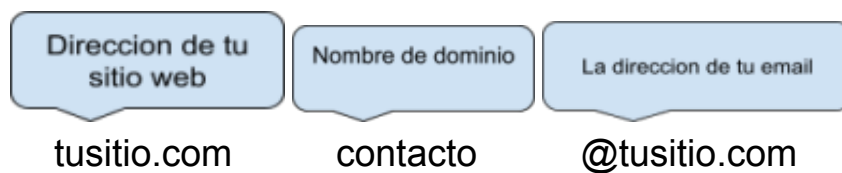


Dominios

¿Qué es un nombre de dominio?

Un nombre de dominio es una cadena de caracteres que identifica una dirección IP en Internet, permitiendo acceder a un sitio web sin conocer la IP exacta del servidor. También se utiliza para servicios como el correo electrónico. Ejemplo:

Ejemplo:



Esta “traducción” entre nombre e IP es posible gracias al sistema **DNS** (**Domain Name System**).

¿Cómo se interpreta un nombre de dominio?

Ejemplo: miempresa.com.ar

[.ar](#) → Dominio de primer nivel (TLD) territorial

[.com](#) → Segundo nivel (gTLD) generico

[miempresa](#) → Tercer nivel (dominio registrado)

Tipos de nombres de dominio

1. Dominio de primer nivel genéricos(gTLD)

.com: Identifica empresas comerciales

.net: Identifica a empresas de internet y telecomunicaciones

.org: Identifica organizaciones sin fines de lucro.

.info: Orientado a proyectos de información.

BIZ: Orientado a actividades de negocios.

2. Dominio genéricos restringido

Dominio	Tipo	Propósito
.edu	Patrocinado	Instituciones educativas acreditadas.
.gov	Patrocinado	Gobierno de EE.UU
.mil	Patrocinado	Fuerzas Armadas de EE.UU
.int	Patrocinado	Organismos internacionales.
.mobi	Patrocinado	Servicios móviles.
.museum, .travel, .jobs, .coop, .aero, .cat, .asia, .travel	Patrocinado	Uso específico con requisitos.

3. Lista de Dominios Territoriales(ccTLD)

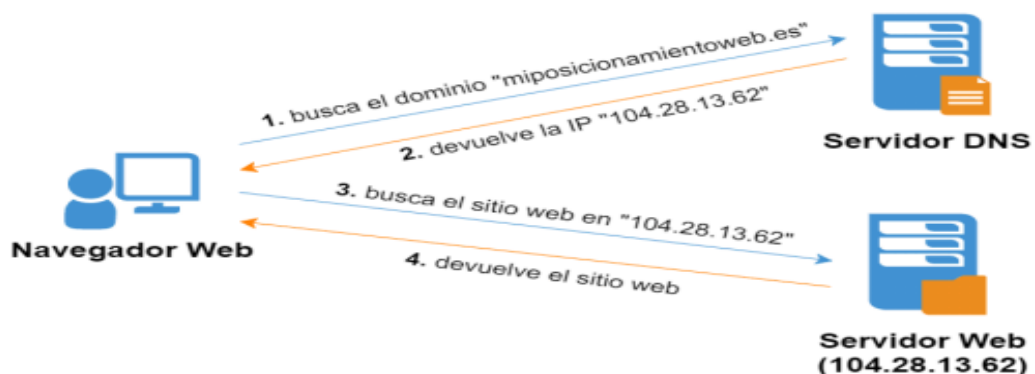
Corresponden a cada país, como: **.ar** (Argentina), **.es** (España), **.br** (Brasil), **.us** (EE.UU), etc.

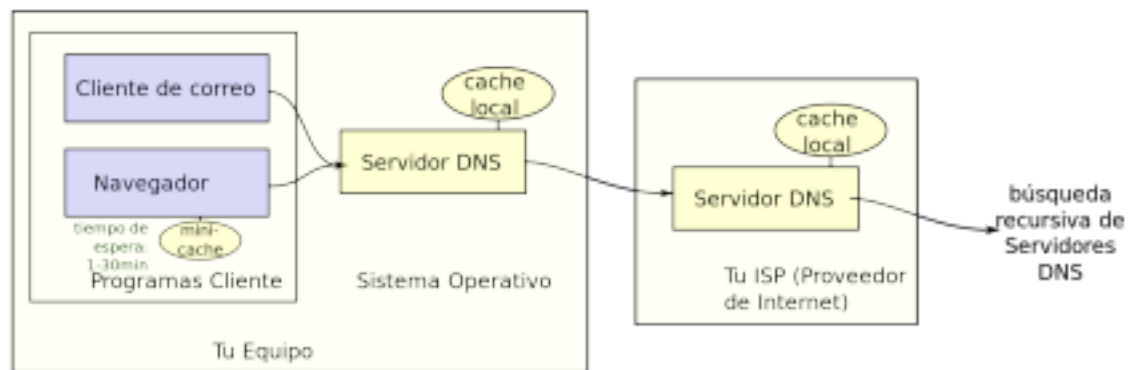
Extensión	País
.ar	Argentina
.es	España
.br	Brazil
.us	EE.UU
.ad	Andorra
.ae	Emiratos Arabes Unidos

.af	Afganistán
.ag	Antigua & Barbuda
.ai	Anguilla
.al	Albania
.am	Armenia
.an	Antillas Holandesas
.ao	Angola
.aq	Antártida
.ar	Argentina
.as	América Samoa
.at	Austria
.au	Australia
.aw	Aruba

¿Cómo funciona el sistema DNS(Sistema de nombres de Dominio)?

El **DNS** es una base de datos distribuida en 13 **servidores raíz** que traduce los nombres de dominio legibles por humanos a direcciones IP numéricas. También permite enrutar correos electrónicos y otros servicios en Internet.





¿Como se adquiere un nombre de dominio?

Depende del tipo de dominio:

Genéricos (gTLD): Se registran a través de entidades acreditados por ICANN. Ejemplo: .com, .org, .net.

Los dominios territoriales(ccTLD): Se registran a través de entidades locales acreditadas (como **NIC Argentina** para .ar).

Requisitos:

gTLD libres: sin requisitos previos.

gTLD restringidos: con requisitos específicos.

ccTLD: según las normativas del país.

Organismos Internacionales

NIC Argentina trabaja conjuntamente con una serie de organismos internacionales.

IANA (Internet Assigned Numbers Authority): Administra direcciones IP y DNS a nivel global.

ICANN (Internet Corporation for Assigned Names and Numbers): Administra los nombres de dominio y numeros IP a nivel global.

LACNIC: Asigna IPs para America Latina y el Caribe.

LACTLD: Es una organización (top-level domain), sin fines de lucro que busca agrupar los ccTLD de América Latina para coordinar políticas y estrategias.

NIC ARGENTINA: Administra el dominio .ar y regula el registro de dominios en Argentina.

Cómo registrar un dominio

Pasos generales (aplica a cualquier registrador):

- **Elegir registrador acreditado** por ICANN o entidad nacional.
- **Comprobar disponibilidad** del nombre de dominio.
- **Verificar disponibilidad legal de marca** si es un nombre comercial.
- **Contratar el dominio y realizar el pago.**
Configurar el panel de control y delegar a los servidores DNS.
- **Renovar** antes del vencimiento para mantener el dominio.

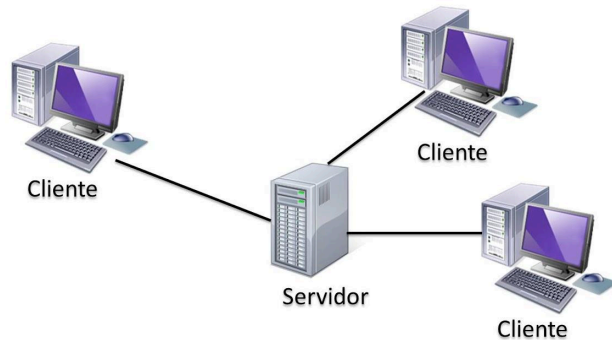
Caso específico: Registro en NIC Argentina (para .ar):

1. **Consultar disponibilidad** desde la web de NIC.ar
 - Si está en uso, se puede activar una alerta de disponibilidad.
 2. **Registrarse como usuario** y luego iniciar sesión.
 3. **Registrar dominio** y añadirlo al carrito.
 4. **Pagar el dominio** mediante la plataforma (tarjeta u otro medio).
 5. **Delegar el dominio** con los DNS de tu proveedor o reservarlo sin delegar.
 - Opción de **autodelegación** para configurar tus propios DNS.
 6. **Confirmar delegación** asociando al menos dos servidores y finalizar.
-

Tipos de servidores

Que es un servidor

Un servidor es una computadora dentro de una red que proporciona servicios o recursos a otras computadoras (clientes). Estos pueden incluir almacenamiento, aplicaciones, sitios web, correo, impresión, etc.



Hardware: Diseñado para tareas intensivas. Permite reemplazar componentes sin apagar el sistema.

Software: Incluye sistemas operativos y aplicaciones especializados para ofrecer servicios estables y seguros

¿Son importantes?

Los servidores hacen posible la mayoría de los servicios digitales actuales. Sin ellos, servicios como correos electrónicos, redes sociales, almacenamiento en la nube o videollamadas y más. También automatizan procesos críticos, como control de tráfico, cámaras de seguridad, entre otros.

Tipos de servidor

Servidor de Aplicaciones (Application Servers): Ejecuta aplicaciones empresariales accesibles desde la red.

Servidor de Audio/Video (Audio/Video Servers): Estos añaden capacidades multimedia a los sitios web, provee streaming de música y video.

Servidor de Chat (Chat Servers): Estos permiten intercambiar información a una gran cantidad de usuarios ofreciendo la posibilidad de llevar a cabo discusiones en tiempo real.

Servidor de fax: Gestiona faxes de forma digital (envío, recepción y almacenamiento).

Servidores FTP (FTP Servers): Permite subir y descargar archivos mediante protocolo FTP.

Servidores Groupware (Servidores de Colaboración): Provee a los usuarios, grupos y organizaciones las herramientas para colaborar se entre ellos. Por ejemplo agendas, calendarios, tareas, etc.

Servidores IRC (IRC Servers): Permiten comunicación en tiempo real entre usuarios.

Servidores Web (Web Servers): Aloja sitios web y los entrega a los usuarios vía HTTP.

Servidores de Noticias (News Servers): Distribuye contenido de grupos de noticias públicos a través de la red de noticias (USENET).

Servidores Proxy (Proxy Servers): Filtra, administra y gestiona el acceso a internet y mejora la seguridad.

Servidor de archivos: Tiene una gran cantidad de almacenamiento, almacena y distribuye archivos a los usuarios en red

Servidor de Base de Datos (database server): Administra bases de datos y responde a consultas.

Servidores de Listas (List Servers): Gestiona listas de distribución de correos.

Servidor de impresiones: Controla una o más impresoras y administra trabajos de impresión de usuarios conectados.

Servidor de correo: Administra el envío, recepción y almacenamiento de correos electrónicos.

Servidor de telefonía: Administra llamadas, VoIP y sistemas de respuesta automática.

Servidor de reserva: Actúa como respaldo ante fallos de otros servidores y tiene un software de reserva de la red instalado permitiendo tener gran cantidad de almacenamiento en discos duros.

Servidor dedicado: Exclusivo para un solo cliente o aplicación. Le dedican toda su potencia a administrar los recursos de la red.

Servidor no dedicado: Son aquellos que no dedican toda su potencia a los clientes, comparten recursos entre funciones de servidor y usuario local.

Servidores Telnet (Telnet Servers): Permite acceso remoto por línea de comandos a otros equipos por línea de comandos.

Servidores Introducción a Servidores Web en la nube

El alojamiento web: Es el servicio que provee a los usuarios de Internet un sistema para poder almacenar información, imágenes, vídeo, etc, permitiendo publicar un sitio o aplicación en Internet. La información se almacena en un servidor web conectado permanentemente.

Tipo de alojamiento en Internet:

- Alojamiento gratuito: Limitado en espacio y funciones, incluye publicidad. Útil para pruebas o sitios personales con poco tráfico.
- Alojamiento compartido: varios sitios en un mismo servidor. Es económico, pero menos seguro y estable. Debido a la reducción de costos al compartir un servidor con cientos miles o millones de personas o usuarios.

- Alojamiento de imágenes: Servicio específico para guardar imágenes, normalmente gratuito y sostenido por publicidad.
- Alojamiento revendedor: Permite revender espacios de alojamiento. Pensado para quienes administran múltiples sitios, está diseñado para grandes usuarios o personas que venden el servicio de hospedaje a otras personas. Tiene muchos dominios y gran cantidad de almacenamiento.
- Alojamientos virtuales: Servidor físico dividido en entornos virtuales. Ofrece mayor control y flexibilidad. Ideal para programadores o empresas pequeñas.
- Servidores dedicados: Servidor entero para un solo cliente. Alto rendimiento, recomendado para sitios con muchas visitas.
- Hosting Administrado: El proveedor se encarga de mantener, configurar y solucionar problemas del servidor (soporte técnico, errores de scripts, CMS, etc.). Ideal para quienes no tienen conocimientos técnicos. Tiene un costo más alto.
- Hosting no Administrado: El proveedor solo ofrece conectividad, recursos y panel de control. Toda la gestión del servidor (instalaciones, actualizaciones, errores) corre por cuenta del cliente. Es más económico, pero requiere experiencia.
- Colocación – housing: Es un servicio donde el cliente lleva su propio servidor físico al centro de datos del proveedor. La empresa solo provee energía, refrigeración y conexión a Internet.
 - ❖ **El servidor es propiedad del cliente**, incluyendo el hardware y configuración.
 - ❖ **Ventaja:** Control total del equipo y configuración.
 - ❖ **Uso común:** Empresas con requerimientos específicos que no quieren usar infraestructura del proveedor.

Alojamiento en la Nube

- **Cloud Hosting:** Es un tipo de alojamiento web basado en tecnologías avanzadas que permiten a múltiples servidores interconectados funcionar como un único sistema. Esta arquitectura distribuye los recursos y la carga de trabajo, garantizando mayor flexibilidad, escalabilidad en tiempo real y alta disponibilidad. A diferencia del hosting tradicional, la seguridad y estabilidad están respaldadas por varios servidores, lo que reduce el riesgo de fallos y elimina limitaciones físicas, permitiendo crecer según la demanda.

Formas de obtener el Hosting

El hosting puede obtenerse de forma gratuita o paga:

- **Servicio pago:** El servicio pago suele contratarse a través de proveedores de internet o empresas especializadas que ofrecen almacenamiento web con servicios adicionales como tareas automatizadas o cuentas de correo. Estos planes suelen pagarse por medios electrónicos como tarjetas o PayPal.
- **Servicio gratuito:** El servicio gratuito no tiene costo, pero presenta limitaciones como menor espacio, acceso restringido a configuraciones y publicidad insertada por el proveedor. Aun así, puede ser útil para pruebas o sitios con poco tráfico.

Servidores y servicios: Un alojamiento web se puede diferenciar de otro por el tipo de sistema operativo, bases de datos y motor de generación de sitios web que exista en él. La combinación más conocida y extendida es la del tipo LAMP (Linux, Apache, MySQL y PHP), aunque se está comenzando a usar una combinación con Java. Los servicios más comunes que se pueden incluir en un hosting son los siguientes:

1. Alojamiento de ficheros y acceso vía web a los ficheros para subidas, descargas, edición, borrado, etc.
2. Acceso a ficheros vía FTP.

3. Creación de bases de datos, típicamente MySQL en el caso de alojamientos basados en Linux y administración vía web de las bases de datos con herramientas web como phpMyAdmin.
4. Cuentas de correo electrónico con dominio propio, gestión de listas de correo, acceso vía clientes de sobremesa (tipo MS Outlook, etc.) y acceso vía webmail a estas cuentas. Reenvía del correo a otras cuentas (incluso externas).
5. Discos duros virtuales que se pueden configurar como unidad de red en un equipo local vía protocolos como WebDav.
6. Copias de seguridad.
7. Gestión de dominios y subdominios.
8. Estadísticas de tráfico.
9. Asistentes para la instalación rápida de paquetes software libre populares como WordPress, Joomla, etc.

Calidad de Servicio y Disponibilidad del Hosting: La disponibilidad total (100%) de un servicio de hosting es casi imposible de garantizar. Por eso, se mide como un porcentaje que refleja el tiempo de actividad del servidor. Cuanto más alto el porcentaje, menor es el tiempo de caída, y esto generalmente influye en el precio del servicio. Es un indicador clave de la calidad del proveedor de hosting.

¿Qué es la Computación en la Nube?

La **Computación en la Nube** (Cloud Computing) es un modelo que permite acceder a recursos y servicios informáticos (como servidores, almacenamiento, redes o software) a través de Internet, **cuando se necesitan y sin preocuparse por la infraestructura física.**

Otra forma de decirlo:

Es como alquilar poder de cómputo en lugar de comprarlo. Se accede por Internet y se paga solo por lo que se usa, igual que con la luz o el agua.

Costos Operacionales:

- Actualización de hardware.
- Parches y mantenimiento de software.
- Tareas de mantenimiento físico(preventivo, correctivo).

Diferencias entre el modelo tradicional y la nube

	Modelo Tradicional	Cloud Computing
Compra	Compra de hardware y software propios	Compra de servicios en la nube
Pago	Gasto fijo por infraestructura	Pago variable según el uso
Acceso	Solo desde la red interna	Desde cualquier lugar vía Internet
Técnico	Infraestructura fija, limitada	Escalable, compartida, flexible

Computacion en la Nube ventajas y desventajas

Ventajas	Desventajas
<u>Reducción de Costos:</u> No se necesita comprar servidores o licencias de software, se paga solo por uso.	<u>Dependencia de Internet:</u> Si no hay conexión, no se puede acceder a los servicios.
<u>Escalabilidad y Velocidad:</u> Se pueden aumentar o reducir recursos rápidamente según la necesidad.	<u>Dependencia del Proveedor:</u> Se depende del proveedor para el buen funcionamiento y soporte.
<u>Innovación en la Tecnología:</u> El proveedor actualiza los recursos, el usuario accede a lo último sin gestionarlo.	<u>Seguridad:</u> Existe el riesgo de exposición de datos confidenciales a terceros.
<u>Ubicación Flexible:</u> Los proveedores pueden ubicar su infraestructura en lugares más económicos..	<u>Rendimiento:</u> Si muchos usuarios comparten la nube, podría disminuir la calidad del servicio.
<u>Accesibilidad:</u> Se puede acceder desde cualquier dispositivo con conexión a Internet.	<u>Escalabilidad:</u> A largo plazo, a medida que mas usuarios empiecen a compartir la infraestructura de la nube, lo que podría llevar a la degradación en el servicio.

Tipos y Modelos de Nube

Existen diversos tipos de nube dependiendo de las necesidades de cada empresa, el modelo de servicio ofrecido y la implementación de la misma, pero básicamente existen tres grandes grupos:

La Nube Pública: Es aquella cuya infraestructura pertenece a un proveedor externo, quien se encarga de administrar, mantener y actualizar los recursos. Los usuarios acceden a los servicios a través de Internet y pagan por lo que utilizan, sin necesidad de preocuparse por la infraestructura.

La Nube Privada: Es una infraestructura dedicada exclusivamente a una organización, la cual la gestiona internamente. Aunque ofrece mayor control y seguridad, también implica mayores costos y menor escalabilidad que la nube pública.

La Nube Híbrida: Combina ambas modalidades: permite a las empresas mantener aplicaciones críticas en su nube privada, mientras aprovechan la nube pública para tareas menos sensibles o para escalar recursos de forma flexible cuando sea necesario

Modelos de servicios en la nube

Existen tres modelos principales de servicios en la nube: **IaaS**, **PaaS** y **SaaS**, cada uno con distinto nivel de control y uso.

1. Infraestructura como un servicio (IaaS): Brinda recursos básicos como servidores, almacenamiento, redes y sistemas operativos a través de Internet. El usuario tiene control sobre la infraestructura y puede escalar según necesidad. Es ideal para empresas que desean gestionar sus propias aplicaciones y sistemas sin comprar hardware físico.

Ejemplos:

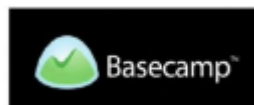


GOGRID

Permite alquilar los recursos tales como servidores, software, redes, memoria, capacidad de procesamiento y almacenamiento, del proveedor en lugar de tener que comprarlos. Adicionalmente, la infraestructura necesaria puede ser escalable dinámicamente en base a los recursos necesarios para la aplicación.

2. Plataforma como un Servicio (PaaS): Ofrece todo lo necesario para soportar el ciclo de vida para desarrollar, probar y desplegar aplicaciones. Incluye herramientas de desarrollo, bases de datos y middleware. Está pensado para desarrolladores, ya que facilita el trabajo sin preocuparse por la infraestructura.
3. Software como un Servicio(SaaS): Proporciona aplicaciones listas para usar, accesibles desde Internet. El proveedor se encarga de todo: infraestructura, mantenimiento y actualizaciones. Es ideal para usuarios finales que solo necesitan usar el software, como correo electrónico, CRM o herramientas de oficina.

Ejemplos:



Computación en la Nube Virtualización (VM)

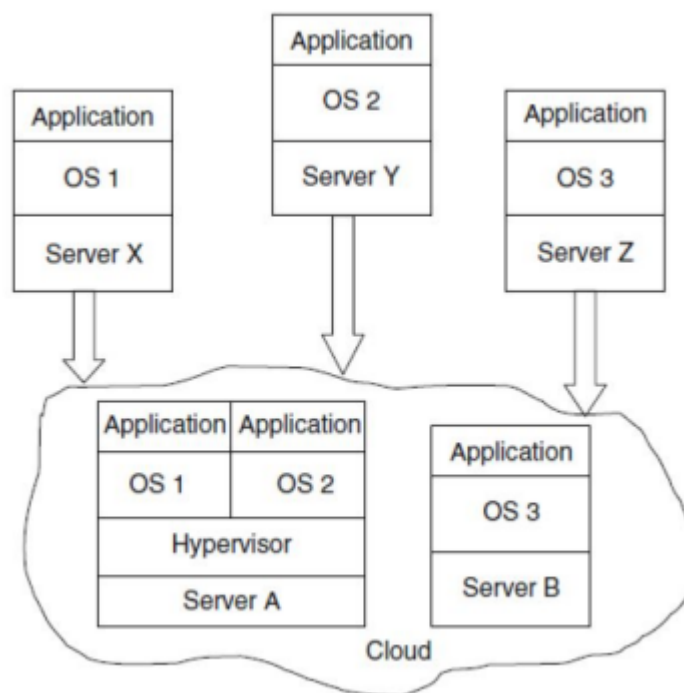
La **virtualización** es una tecnología que permite **crear versiones virtuales** de recursos físicos como servidores, almacenamiento o redes. En lugar de usar hardware físico para cada aplicación, se pueden ejecutar **múltiples sistemas en una sola máquina** física, administrados por un programa especial llamado **hipervisor**.

- Una infraestructura virtualizada es la base para la mayoría de las nubes de alto rendimiento.
- Se utiliza de manera amplia para poner en común los recursos de la infraestructura y puede proporcionar los elementos básicos para mejorar la agilidad y flexibilidad de un sistema en nube.
- Los servidores continúan siendo el foco principal de la virtualización.

Se refiere a la abstracción de los recursos lógicos lejos de sus recursos físicos subyacentes.

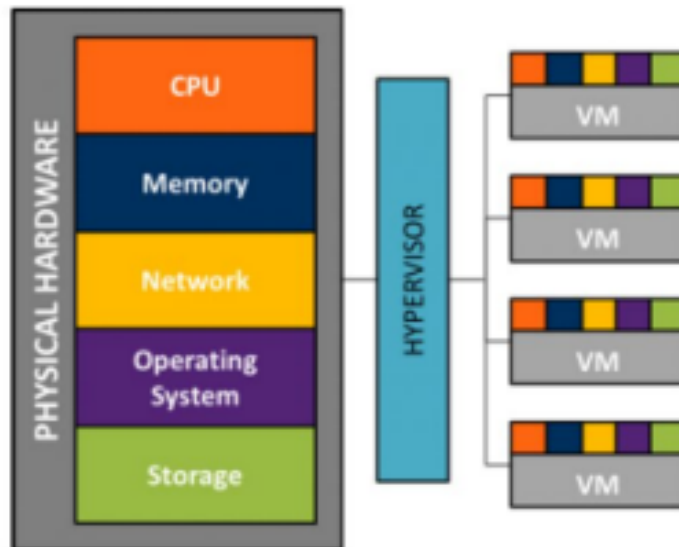
La virtualización no es necesaria para crear un entorno en nube, pero permite una rápida escalabilidad de los recursos de una manera que en entornos no virtualizados se encuentra difícil de alcanzar. La ventaja de la computación en nube es la capacidad de virtualizar y compartir recursos entre las diferentes aplicaciones con el objetivo de mejorar la utilización del servidor

Ejemplo de virtualización:



La virtualización permite que múltiples instancias de los recursos de la infraestructura se ejecuten en el mismo hardware, con el control de acceso a esos recursos controlados por un hipervisor.

Hipervisor (VMM - Virtual Machine Manager)



Es el software que **administra las máquinas virtuales (VM)**. Controla el acceso al hardware físico y permite ejecutar varios sistemas operativos simultáneamente.

Existen hipervisores:

- Nativos (unhosted o bare metal) :Se instala directamente sobre el hardware físico. Es más rápido y eficiente.
- Anfitriones (hosted):Se ejecuta sobre un sistema operativo, y dentro de él se corren las máquinas virtuales.

Tipos de Virtualización

De servidores: Ejecutar varios servidores virtuales en un solo equipo físico.

De almacenamiento: Agrupar dispositivos físicos de almacenamiento en uno solo virtual.

De red: Crear redes virtuales independientes de la infraestructura física.

Movilidad de Recursos

Patrones que permiten mover recursos entre entornos físicos y virtuales:

- P2V (Physical to Virtual): De físico a virtual.
- V2V (Virtual to Virtual): De una VM a otra VM.
- V2P (Virtual to Physical): De virtual a físico.
- P2P (Physical to Physical).
- D2C (Datacenter to Cloud): Entre nubes o centros de datos.
- C2C (Cloud to Cloud): Entre nubes o centros de datos.
- C2D (Cloud to Datacenter): Entre nubes o centros de datos.
- D2D (Datacenter to Datacenter): Entre nubes o centros de datos.

Beneficios de la virtualización

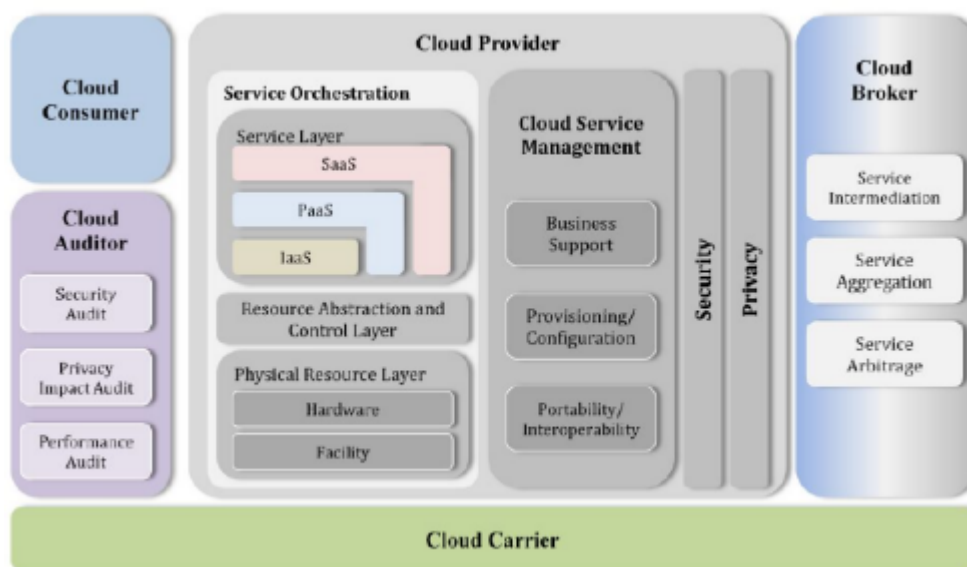
- Reducción de costos al usar menos hardware.
- Facilita la administración y mantenimiento.
- Mayor disponibilidad y confiabilidad.
- Permite tercerizar servicios IT fácilmente.
- Escalabilidad rápida según necesidades.
- Mejora la calidad del servicio (QoS).

Retos de la virtualización

- Puede haber problemas de rendimiento.
- Riesgos en seguridad y privacidad.
- Mayor complejidad en el control.
- Costos de ancho de banda pueden aumentar.
- Confiabilidad del proveedor debe ser alta.

Arquitectura de Referencia de Computación en Nube del NIST

El **NIST (National Institute of Standards and Technology)** define una arquitectura conceptual de la nube que identifica a los principales **actores, sus funciones y actividades**. Esta arquitectura facilita la comprensión de los **requisitos, usos, características y estándares** de la computación en la nube.



Portabilidad de la Información: La portabilidad consiste en la capacidad del cliente para trasladar sus datos y aplicaciones entre distintos proveedores de nube o hacia su propia infraestructura, sin interrupciones ni pérdida de acceso.

- **Objetivo:** Mitigar riesgos e incrementar la flexibilidad.
- Una solución se considera abierta si facilita este proceso y garantiza la continuidad del servicio.

Retos y Riesgos del Cloud Computing

Aunque la nube ofrece muchas ventajas, también conlleva **riesgos específicos**:

1. Falta de Transparencia:

- ¿Dónde están los datos? Ubicación geográfica desconocida.
- Accesos privilegiados: Personas con altos permisos podrían acceder indebidamente a la información.
- Aislamiento insuficiente: Los datos comparten infraestructura con otros clientes.
- Subcontratación: Terceros pueden gestionar partes del servicio sin control directo del cliente.

2. Falta de Control:

- Recuperación: Falta de planes de contingencia o copias distribuidas.
- Soporte investigativo: Dificultad para rastrear actividades ilegales por la dispersión de logs y datos.
- Viabilidad a largo plazo: Posibles cambios en el proveedor (venta, quiebra, fusión) afectan el servicio.

Seguridad en Cloud Computing:

- Integridad de los datos: Es necesario poder asegurar que los datos no han sido modificados por entidades no autorizadas, y que la información es la original.
- Disponibilidad de la información: El usuario debe poder acceder a la información cuando la necesite, pese a fallos del proveedor.
- Confidencialidad o privacidad: Es la garantía de que sólo puedan acceder a los datos de los usuarios autorizados. El usuario, en principio, no tiene control completo del acceso del proveedor a sus datos.

Cloud Computing y Legislación

- Ley de protección de los datos: Varía según el país.
- **Responsabilidad:** Puede recaer en el proveedor, la organización contratante o el propietario de los datos.
- **Diversidad legislativa:** Es clave entender qué leyes se aplican, especialmente si los datos cruzan fronteras.
- Considerar la Jurisdicción.
- Evaluar la aplicabilidad de disposiciones nacionales e internacionales.
- Recolectar y revisar los requisitos legales normativos necesarios para ambas partes.
- Evaluar si los términos del proveedor cumplen los requisitos del cliente.
- Estipular todo lo necesario en los SLA.
- Analizar el sistema de penalizaciones.

Cloud

Servicios de la Nube: Cuadro comparativo de modelos de servicios, componentes y responsables

MODELO DE SERVICIOS	STACK DE LA NUBE	COMPONENTES DEL STACK	RESPONSABLES		
SAAS PAAS IAAS	USUARIO	Login Registro Administración	Cliente	Cliente	Cliente
	APLICACIÓN	Autenticación Interfaz Usuario Reportes Autorización Transacciones Dashboard			Vendedor
	STACK DE APLICACIÓN	OS App Srv Database Lenguaje Programación Middleware Monitoreo			
	INFRAESTRUCTURA	Data Center Servidores Red Almacenamiento Firewall Balanceador carga	Vendedor	Vendedor	

Niveles de responsabilidad proveedor Cloud vs consumidor Cloud:

Modelo de entrega de Cloud	Actividades comunes de consumidores de Cloud	Actividades comunes del proveedor de Cloud
SaaS(Software como Servicio)	Uso y configuración de un servicio de Cloud	Implementa, mantiene y administra toda la app.
PaaS(Plataforma como Servicio)	Desarrolla, prueba e implementa sus aplicaciones.	Proporciona la plataforma, middleware y recursos necesarios.
IaaS(Infraestructura como Servicio)	Instala, configura y gestiona el software sobre la infraestructura.	Administra el hardware, almacenamiento, redes y recursos base.

Entre los principales sistemas operativos para nubes existen en el mercado ,tenemos los siguientes:

Software abierto

- OpenStack: es un sistema operativo de nubes desarrollado por **Rackspace y la NASA**, permite personalización y escalabilidad. Licencia Apache 2.0.
- CloudStack: plataforma para crear y gestionar servicios de nube.
- Eucalyptus: se integra con AWS, orientado a nubes privadas.

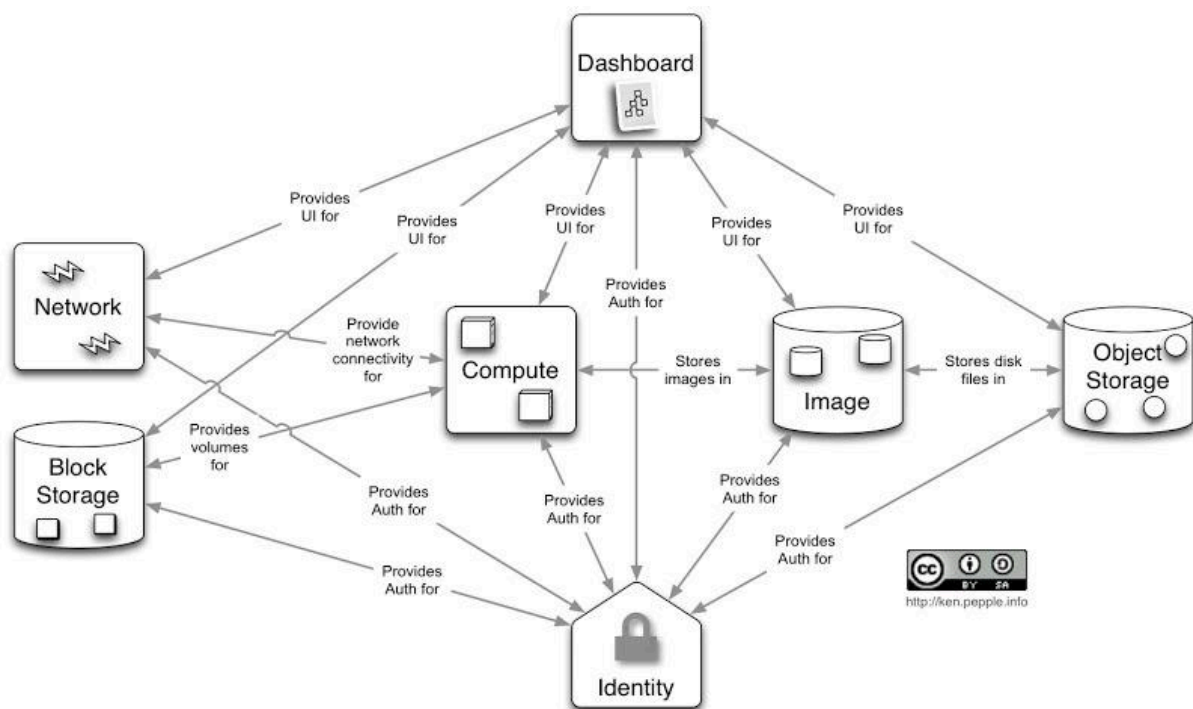
Software cerrado

- Vmware Vcloud
- Microsoft Azure
- Red Hat RHLE

Arquitectura de OpenStack

OpenStack se compone de **módulos o proyectos**, cada uno con una función específica, que trabajan juntos usando **APIs** (interfaces que permiten que se comuniquen entre sí y se comparten los recursos).

Diagrama conceptual: de lo que se logra a través de las APIs y del fin en conjunto de los componentes de OpenStack.



Módulos principales

Horizon: Es el módulo de Dashboard o panel de control o panel web de control para gestionar todo y provee una interfaz de usuario.

Neutrón: Es el módulo de red administra la red.

Keystone: Es el módulo de identidad y se encarga del sistema de autenticación y permisos.

Glance: Es el módulo de imagen Manejo de imágenes de máquinas.

Swift: Almacenamiento de objetos (como Dropbox).

Cinder: Almacenamiento en bloques (como un disco duro virtual).

¿Qué es AWS?

Amazon Web Services (AWS) es una **plataforma de servicios en la nube** ofrecida por Amazon. Permite a empresas y desarrolladores acceder a servicios como servidores, almacenamiento, bases de datos y redes **a través de Internet y bajo demanda**, sin necesidad de mantener infraestructura física propia.



Ventajas de AWS vs TI tradicional

Flexibilidad: Podés **escalar** los recursos automáticamente (aumentar o reducir) según lo que necesites en el momento.

Pago por uso: Solo pagás por lo que usás, como un servicio (gasto operativo), no como una compra de equipos (gasto de capital).

Agilidad: Podés configurar, actualizar o cerrar recursos cuando lo necesites, sin demoras ni grandes inversiones.

Automatización: Todo se puede gestionar con herramientas web o scripts, sin necesidad de intervención manual.

Categorías de los servicios de AWS

Los servicios de AWS se clasifican en diferentes categorías y cada una contiene uno o varios servicios.

Ejemplo de una solución sencilla elastic cloud: Imagina que estás desarrollando una **aplicación de base de datos**. Podés usar servicios de AWS así:

1. **Amazon EC2:** Ejecutas servidores virtuales que reciben los datos de los usuarios.
2. **Amazon S3:** Almacenás los objetos o archivos de cada usuario.
3. **Amazon DynamoDB:** Usás una base de datos NoSQL para buscar y organizar esos datos.
4. **Amazon VPC:** Aislás toda tu infraestructura en una red privada y segura dentro de la nube.

Otros Servicios de Cómputo en AWS (según necesidades)

Amazon EC2: Control total de máquinas virtuales.

AWS Lambda: Ejecutar código sin gestionar servidores.

AWS Elastic Beanstalk: Desplegar apps web sin preocuparse por la infraestructura.

Amazon Lightsail: Plataforma simple para apps pequeñas

AWS Batch: Ejecutar grandes cargas de trabajo por lotes.

AWS Outposts: Usar infraestructura AWS en tu propio centro de datos.

Amazon ECS, EKS y Fargate: Implementa y ejecuta una arquitectura de microservicios

VMware Cloud on AWS: Dispone de una plataforma de virtualización

Existen 3 formas de interactuar con AWS

Consola de administración de AWS: Interfaz gráfica accesible desde el navegador. Permite gestionar la mayoría de los servicios de AWS fácilmente.

(Nota: Las funciones nuevas pueden no estar completamente disponibles al principio).

Interfaz de línea de comandos de AWS (CLI): La CLI de AWS ofrece un **conjunto de utilidades que se pueden lanzar desde un script de comandos** en Linux, macOS o Microsoft Windows.

Kits de desarrollo de software (SDK): Bibliotecas que permiten integrar AWS con aplicaciones en distintos lenguajes (como Python, Java, JavaScript). Útil para programadores.

CAF Cloud Adoption Framework (Marco de adopción de la nube)

Marco de adopción de la nube de AWS
(CAF de AWS)

aws academy

NEGOCIO	PLATAFORMA
GENTE	SEGURIDAD
GOBERNANZA	OPERACIONES

Perspectivas del CAF de AWS

- El CAF de AWS ofrece orientación y prácticas recomendadas a fin de ayudar a las organizaciones a crear un enfoque integral para la informática en la nube, en toda la organización y durante todo el ciclo de vida de la TI, para **acelerar la adopción satisfactoria de la nube**.
- El CAF de AWS está organizado en **seis perspectivas**.
- Las perspectivas se componen de conjuntos de **capacidades**.

© 2019 Amazon Web Services, Inc. o sus empresas afiliadas. Todos los derechos reservados.

52

Es un **modelo creado por AWS** para ayudar a las organizaciones a

planificar su migración a la nube, identificando debilidades en procesos, habilidades y estructuras.

Está organizado en **6 perspectivas**, cada una centrada en un área clave del negocio:

PERSPECTIVAS

1. **Negocio:** Desde la perspectiva de los negocios (por ejemplo, gerentes comerciales, directores de finanzas, responsables del presupuesto y partes interesadas en la estrategia) pueden utilizar el CAF de AWS con el fin de crear un argumento empresarial sólido para la adopción de la nube y de priorizar las iniciativas de adopción de la nube.

Las partes interesadas deben asegurarse de que las estrategias y los objetivos empresariales de una organización estén en consonancia con las estrategias y objetivos de la TI.

Alinea los **objetivos empresariales con los de TI**.

Ayuda a **justificar la adopción de la nube** desde lo económico y estratégico.

Dirigido a: gerentes, directores financieros, responsables de presupuestos.

2. **Personal:** Desde la perspectiva del personal (por ejemplo, directores de recursos humanos y personal) pueden utilizar el CAF de AWS para evaluar funciones y estructuras organizativas, requisitos de habilidades y procesos nuevos y, así, identificar deficiencias. Realizar un análisis de las necesidades y las deficiencias puede ayudar a priorizar la formación técnica, la dotación de personal y los cambios organizativos para crear una organización ágil. Evalúa las habilidades y estructuras organizativas actuales.

Ayuda a detectar necesidades de formación y preparar al equipo para la nube.

Dirigido a: RRHH, líderes de equipo, directores de talento.

3. Gobernanza: Desde la perspectiva de la gobernanza (por ejemplo, el director de información, los directores de programas, los arquitectos empresariales, los analistas de negocios y los administradores de portafolios) pueden utilizar el CAF de AWS para concentrarse en las habilidades y los procesos necesarios para poner en consonancia la estrategia y los objetivos de la TI con la estrategia y los objetivos empresariales.

Este enfoque ayuda a la organización a maximizar el valor de negocio de su inversión en TI y minimizar los riesgos comerciales.

- Asegura que la TI esté alineada con la estrategia del negocio.
- Mejora el valor de la inversión en tecnología y reduce riesgos.
- Dirigido a: CIOs, analistas de negocio, arquitectos empresariales.

4. Plataforma: Desde la perspectiva de la plataforma (por ejemplo, director de tecnología, directores de TI y arquitectos de soluciones) utilizan una variedad de dimensiones y modelos arquitectónicos para comprender y comunicar la naturaleza de los sistemas de TI y sus relaciones. Deben poder describir la arquitectura del entorno de estado de destino detalladamente. El CAF de AWS incluye principios y patrones para la implementación de nuevas soluciones en la nube y para la migración de cargas de

trabajo de las instalaciones a la nube.

- Se enfoca en la **arquitectura técnica** y cómo migrar cargas de trabajo a la nube.
- Proporciona principios y patrones para diseñar soluciones escalables.
- Dirigido a: arquitectos de soluciones, CTOs, líderes de tecnología.

5. Seguridad:

- Establece controles para asegurar **confidencialidad, integridad y visibilidad**.
 - Facilita el cumplimiento de normativas y el control de accesos.
 - Dirigido a: CISOs, responsables de seguridad, auditores.
-

6. Operaciones

- Define cómo se gestionarán los sistemas una vez en la nube.
- Ayuda a adaptar los procesos operativos y a formar al equipo de soporte.

Dirigido a: responsables de soporte, jefes de operaciones TI.

Perspectivas

Seguridad: Se enfoca en que la organización cumpla con los requisitos de seguridad, como:

visibilidad: saber qué ocurre en el entorno.

Auditoría: registrar actividades y accesos.

Control: definir quién puede hacer qué.

Agilidad: adaptarse rápidamente a nuevas amenazas o requisitos.

¿Quiénes la usan?

Responsables de seguridad de TI, como CISOs, administradores y analistas de seguridad.

¿Para qué sirve?

Para diseñar e implementar controles de seguridad que se ajusten a las necesidades de la organización en su paso a la nube

Operaciones: Se enfoca en cómo se deben ejecutar y mantener las operaciones del negocio en la nube, incluyendo tareas diarias, mensuales y anuales.

¿Quiénes la usan?

Directores de operaciones y soporte de TI.

¿Para qué sirve?

- Adaptar procesos operativos al entorno en la nube.
- Definir procedimientos claros.
- Identificar formación técnica necesaria y cambios en procesos.

CLOUD - ARQUITECTURA

¿Qué es la arquitectura de nube?

La arquitectura de nube define cómo se integran y conectan las tecnologías que forman una nube. No se trata solo del hardware (infraestructura), sino del diseño técnico completo que permite ofrecer servicios en la nube.

Incluye:

- Contenedores

- Coordinación
- APIs (interfaces de programación)
- Enrutamiento
- Seguridad
- Automatización
- Gestión
- Experiencia de usuario (UX)

Diferencia clave:

Infraestructura de nube = componentes físicos y virtuales (servidores, redes, etc.).

Arquitectura de nube = cómo se organizan y conectan esos componentes.

Arquitecturas públicas, privadas, híbridas y multicloud

Arquitectura de nube pública: Es un entorno de computación en la nube creado a partir de recursos que no pertenecen al usuario final. Estos recursos, como servidores y almacenamiento, son propiedad de un proveedor externo que también se encarga de su gestión. Los servicios son compartidos por múltiples clientes (inquilinos) y se accede a ellos a través de una interfaz de autoservicio. Este modelo es altamente escalable, permite implementar soluciones rápidamente y es ideal para organizaciones que necesitan flexibilidad sin invertir en infraestructura propia.

Arquitectura de nube privada: La arquitectura de nube privada está diseñada exclusivamente para un solo usuario o empresa. Funciona generalmente dentro del firewall del cliente, aunque también puede alojarse en centros de datos externos alquilados. En este modelo, toda la infraestructura de TI está completamente aislada, lo que permite un mayor nivel de control, seguridad y personalización. Se utiliza cuando se

requiere un entorno dedicado, con políticas estrictas de gestión de datos o cumplimiento normativo.

Beneficios:

- Mayor capacidad de infraestructura para manejar grandes demandas informáticas y de almacenamiento.
- Acceso a servicios bajo demanda mediante interfaces de autoservicio y gestión basada en políticas.
- Asignación eficiente de recursos según las necesidades del usuario.
- Mayor visibilidad y control sobre los recursos de la infraestructura.

Arquitectura de nube híbrida: La arquitectura de nube híbrida combina distintos entornos de nube, como una nube pública y una privada, o incluso varias nubes del mismo tipo (públicas o privadas) conectadas entre sí. Su principal ventaja es que permite coordinar y mover cargas de trabajo entre entornos según la necesidad, logrando así mayor flexibilidad, continuidad operativa y optimización de recursos.

Hoy en día, tanto las nubes públicas como privadas pueden estar alojadas dentro o fuera de las instalaciones del cliente. Por eso, la nube híbrida no se define por su ubicación, sino por su funcionalidad: conectar sistemas, consolidar recursos, escalar horizontalmente, mover cargas de trabajo, gestionar todo desde un solo panel y automatizar los procesos.

Características esenciales de la nube híbrida:

- Conectividad entre distintos sistemas.
- Consolidación y escalado horizontal de recursos.
- Migración fluida de cargas de trabajo entre entornos.
- Gestión unificada desde una herramienta central.
- Automatización de procesos y tareas de TI.

Combinaciones posibles de entornos en una nube híbrida:

- Al menos una nube privada y una pública
- Dos o más nubes privadas
- Dos o más nubes públicas
- Un entorno virtual o sin sistema operativo conectado a una nube pública o privada.

Arquitectura multicloud: Se refiere al uso de dos o más servicios de nube (públicos o privados), generalmente proporcionados por distintos proveedores. A diferencia de la nube híbrida, no siempre hay integración entre ellos. Multicloud implica la coexistencia de múltiples servicios de nube independientes, que pueden o no estar conectados entre sí. Esto permite a las empresas aprovechar lo mejor de cada proveedor para distintos aspectos del negocio.

Cuál es la Diferencia entre el entorno multicloud y la nube híbrida?

Nube híbrida	Multicloud
Combina distintos tipos de nube (pública + privada).	Usa varias nubes del mismo tipo pero de distintos proveedores.
Hay conexión y coordinación entre ellas.	No siempre están conectadas ni integradas.

Ventajas de las nubes múltiples

Flexibilidad	Proximidad	Conmutación por error
Es posible encontrar la solución de nube perfecta para cada necesidad de la empresa. Por ejemplo, una nube propietaria puede ser ideal para alojar	Para mejorar el tiempo de respuesta de usuarios que están lejos de la sede principal de la empresa, se pueden usar proveedores	El entorno multicloud permite mantener copias de seguridad disponibles y escalables. Si una nube falla, otra puede tomar su lugar

aplicaciones con licencias específicas, mientras que una nube más económica puede servir para almacenar archivos públicos, y otra altamente escalable puede alojar sistemas con uso variable. La combinación de diferentes nubes permite cubrir lo que una sola no puede hacer por sí misma.	de nube más cercanos a esos usuarios. Esto mejora la disponibilidad y también permite cumplir con leyes y regulaciones locales, ya que los datos pueden mantenerse dentro del país o región correspondiente.	automáticamente. Esta redundancia protege a la empresa de interrupciones en el servicio y garantiza la continuidad del negocio, aplicando el principio de “si algo puede fallar, eventualmente fallará” como sugiere la ley de Murphy, para cuando falle.
--	--	---

Esquema para la seguridad de los servicios en la nube

Ciclo de vida de los datos: El ciclo de vida de los datos en la nube abarca desde su creación, almacenamiento y uso, hasta su eliminación definitiva. Cada una de estas fases requiere medidas específicas de seguridad, privacidad y control, ya que los datos pueden estar expuestos en cualquier momento del proceso.

Seguridad en la Computación en la Nube

La computación en la nube permite acceder a recursos desde cualquier dispositivo conectado a Internet. Sin embargo, uno de los mayores desafíos es garantizar la seguridad y la privacidad de los datos en cada fase de su ciclo de vida (creación, uso, almacenamiento y eliminación).

Las nueve amenazas a la seguridad:

1. Violación de datos privados.
2. Pérdida de datos.
3. Secuestro de cuentas o servicios (*hijacking*).
4. Interfaces y APIs inseguras.
5. Ataques de denegación de servicio (DoS)
6. Amenazas internas (insiders).
7. Abuso de servicios en la nube. El uso de recursos.

8. Falta de evaluación previa del proveedor
9. Vulnerabilidades en tecnologías compartidas.

Evaluación de los riesgos de seguridad: antes de contratar un proveedor en la nube es necesario evaluar:

- Control de accesos y privilegios de usuarios.
- Cumplimiento de la normativa.
- Ubicación y separación de los datos
- Recuperación ante desastres.
- Soporte a la investigación.
- Viabilidad a largo plazo del proveedor

Pautas de seguridad y privacidad en nubes públicas

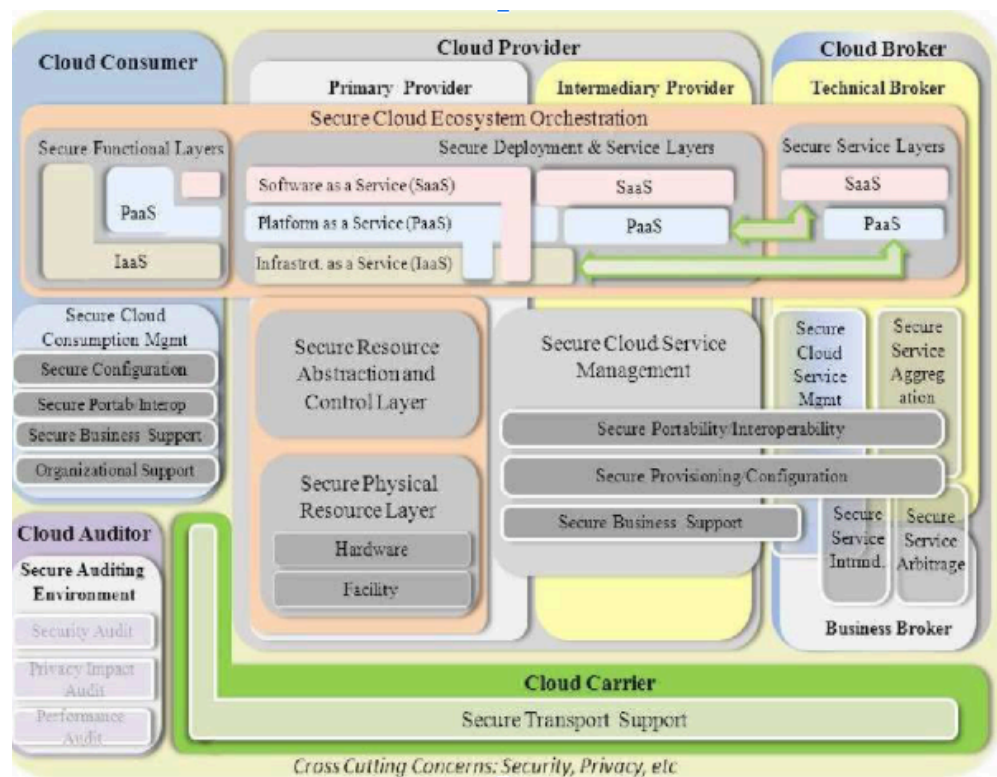
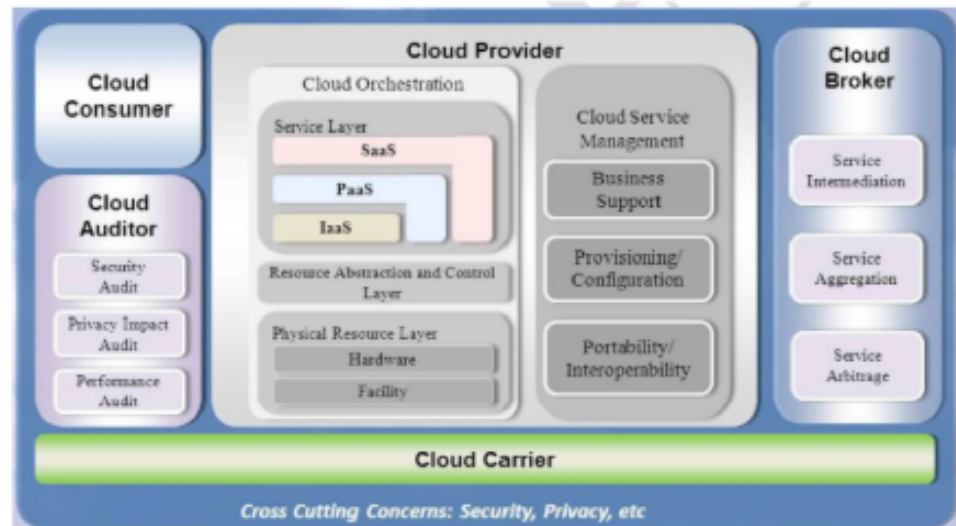
Las cuestiones de seguridad y privacidad que pueden tener impacto a largo plazo en una infraestructura en la nube pública, y en muchos casos, para otros modelos de despliegue son:

1. Gobernabilidad.
2. Cumplimiento.
3. Confianza.
4. Arquitectura.
5. Gestión de la identidad y el acceso.
6. Aislamiento del software.
7. Protección de datos.
8. Disponibilidad.
9. Respuesta a incidentes.

Arquitectura de Referencia de Seguridad del NIST

- Define **componentes de seguridad** según el modelo de servicio (IaaS, PaaS, SaaS) y despliegue (pública, privada, híbrida).
- Establece roles y responsabilidades para cada actor.
- Permite construir un modelo de análisis formal (NCC-SRA) enfocado en seguridad.

Modelo Formal de la NCC-SRA



En primer plano los componentes de la arquitectura de seguridad para cada uno de los actores ilustrados en segundo plano.

El modelo **NCC-SRA** (NIST Cloud Computing Security Reference Architecture) permite analizar formalmente la seguridad en la nube. Este modelo se enfoca en la relación entre los componentes de seguridad y los distintos actores involucrados (cliente, proveedor, auditor, etc.) según el tipo de servicio (IaaS, PaaS, SaaS) y el modelo de implementación (pública, privada, híbrida).

Este enfoque facilita identificar roles, responsabilidades y controles necesarios en cada escenario de uso.

Modelo de Seguridad Compartida de AWS

Amazon Web Services aplica un **modelo de responsabilidad compartida** en seguridad:

Responsabilidad de AWS: En relación con la "seguridad de la nube": AWS es responsable de proteger la infraestructura física y lógica que soporta los servicios en la nube de. Esta infraestructura está conformada por el hardware, el software, las redes y las instalaciones que ejecutan los servicios de la nube de AWS.

Responsabilidad del cliente: El cliente es responsable de proteger y configurar correctamente los recursos que utiliza. Esto incluye la gestión del sistema operativo, software o aplicaciones instaladas, configuración de los firewalls y control de accesos.

Ejemplo: un servicio como Amazon Elastic Compute Cloud (Amazon EC2) se clasifica como Infraestructura como servicio (IaaS) y, requiere que el cliente realice las tareas de administración y configuración de seguridad necesarias.

Los clientes son responsables de la administración del sistema operativo huésped (incluidos los parches de seguridad y las actualizaciones), de cualquier utilidad o software de aplicaciones que el cliente haya instalado en las instancias y de la configuración del firewall provisto por AWS (llamado grupo de seguridad) en cada instancia.

Los riesgos de la información en la nube

Pérdida de Gobernanza: El cliente necesariamente cede al proveedor el control de una serie de cuestiones que pueden influir en la seguridad de sus datos e infraestructura. Al mismo tiempo pueden existir deficiencias en los acuerdos de nivel de servicio (SLA).

- Contratos poco claros
 -
 - SLA (acuerdos de nivel de servicio) incompletos
- Riesgos en la cadena de suministro (subcontratos)

Vinculación: Si no se garantiza la portabilidad del servicio la migración de datos o de tecnologías de un proveedor a otro puede traer serios problemas.

- Migraciones forzadas o mal planificadas
- Quiebra del proveedor.
- Crisis de confianza con el proveedor.
- Portabilidad o Interfaces deficientes.

Cumplimiento: Se deben alcanzar los requisitos legales y normativos, se debe garantizar la posibilidad de efectuar una auditoría de los servicios brindados por el proveedor.

- Poca transparencia en los contratos y subcontratos relacionados.
- Auditoría no disponible para clientes.
- Falta de cumplimiento del proveedor que afecte
- certificaciones del cliente.

Agotamiento de recursos: La infraestructura debe estar bien dimensionada bajo demanda.

- **Sobredimensionamiento:** costos innecesarios.
- **Sobredimensionamiento:** caídas del servicio.
- Pérdida de confianza o daño a la imagen.

Fallos de aislamiento: La nube usa recursos compartidos. Si el aislamiento falla.

- Puede haber acceso no autorizado.
- Riesgos de pérdida o alteración de datos.
- Mal uso de encriptación o autenticación.

Supresión insegura de datos: La eliminación incompleta de datos puede permitir su recuperación futura:

- Imposibilidad técnica del proveedor para efectuar eliminación definitiva
- Migración de Proveedor y reubicación de hardware.
- **Mitigación:** usar encriptación fuerte

Fuga de Datos: Mayor volumen de datos en tránsito o almacenados aumenta el riesgo:

- Ataques MITM(Man in the Middle).
- Falta de cláusulas de confidencialidad o de no divulgación en el contrato.
- Personal malintencionados en los proveedores de nube

Órdenes Judiciales y Jurisdicciones: Los datos pueden estar almacenados en países con leyes de riesgo:

- Confiscación de servidores
- Fallas en el aislamiento
- Jurisdicciones de alto riesgo
- Falta de información sobre ubicación física
- **Recomendación:** conocer dónde están tus datos y la legislación aplicable-

Protección de datos: El cliente sigue siendo el principal responsable ante la ley:

- Infracciones de la ley
- Pérdida de control por parte del cliente
- Virtualización en equipos compartidos
- Procesamiento indebido de datos sensibles cuando el cliente será el principal responsable del procesamiento incluso cuando lo realice el proveedor.
- Publicación de información protegida

Congestión o fallos en la red: El acceso a la nube depende de Internet. Riesgos:

- Desastres naturales
- Ataques de DoS
- Fallos en los dispositivos de red(routers o enlaces físicos).

Ataques de ingeniería social: Es el arte de engañar a las personas para que revelen información sensible como para obtener credenciales o instalar malware.

- Robo de credenciales(identidad).
- Robo de información sensible.
- **Mejor defensa:** capacitar a los usuarios.

Compromiso de los registros (logs): Los registros pueden estar

dispersos o inaccesibles y estan distribuidos en máquinas virtuales del proveedor .

- Falta de acceso a los logs por parte del cliente
- Pérdida de evidencias
- Incumplimientos normativos
- Los registros de log y la gestión de incidentes centralizada minimiza el riesgo.

CONCLUSIONES

- Revisión de Contratos por todas las áreas involucradas
- Revisión de Acuerdos de Servicio (SLA)
- Encriptación de Datos
- Auditoría periódica del proveedor
- Conocer la Jurisdicción de los datos
- Gestionar los Incidentes y logs en forma centralizada
- Mantener un adecuado dimensionamiento de la infraestructura
- Capacitar a nuestros recursos humanos en éste nuevo paradigma

Inteligencia artificial (Cloud IA)

La **Inteligencia Artificial (IA)** es una disciplina de las ciencias de la computación que desarrolla algoritmos capaces de imitar el razonamiento humano. Estos algoritmos se ejecutan tanto a nivel de software como hardware, buscando eficiencia en **tiempo de procesamiento y consumo energético**.

Las 4 etapas de desarrollo

Máquinas recreativas: solo reacciona a diferentes tipos de estímulos basados en reglas preprogramadas.

Memoria limitada: la mayor parte de la IA moderna es de memoria limitada. Puede usar la memoria para mejorar con el tiempo mediante el entrenamiento con datos nuevos, por lo general, a través de una red neuronal artificial o algún otro modelo de entrenamiento.

Teoría de la mente: aún en desarrollo; simula emociones y toma de decisiones como un humano.

Autoconocimiento: hipotética; una IA que entienda su existencia y tenga conciencia propia teniendo las capacidades intelectuales y emocionales de un ser humano.

TIPOS de IA

IA (Inteligencia Artificial): una máquina que es capaz de imitar el razonamiento humano.

ML (Machine Learning): la máquina aprende a partir de datos que le enseñan patrones (requiere entrenamiento de personas).

DL (Deep Learning): una forma avanzada de ML que aprende por sí misma usando redes neuronales profundas, simulando el cerebro humano.

Modelos de entrenamiento de IA

Aprendizaje supervisado: se entrena con datos etiquetados.

Aprendizaje no supervisado: identifica patrones por sí solo en datos sin etiquetar.

Aprendizaje por refuerzo: aprende con prueba y error, como un videojuego.

Aprendizaje automático: nombre general que incluye todos los anteriores.

Tipos de redes neuronales artificiales

Un tipo común de modelo de entrenamiento en la IA es una red neuronal artificial, que se basa a grandes rasgos en el cerebro humano.

1. Feedforward (FF): flujo de datos en una sola dirección.
2. Recurrente (RNN): tiene memoria, útil para datos secuenciales.
3. LSTM: mejora las RNN recordando datos a largo plazo.
4. Convolutacional (CNN): especializada en imágenes y visión computacional.
5. GAN (Generative Adversarial Network): genera contenido nuevo a partir de patrones.

Productos y servicios relacionados Google Cloud

- **Infraestructura IA:**Opciones para que todas las empresas entrenen modelos de aprendizaje .
 - **Vertex AI:**Compila e implementa y escala modelos AA mas rápido con herramientas.
 - **Document AI:**Automatiza la captura de datos en gran escala.
 - **Contact center AI:**Entrega un servicio de atención al cliente de alta calidad y eficiencia.
 - **AutoML:**Entrena modelos de aprendizaje automáticos personalizados y de alta calidad.
 - **Recomendaciones IA:**Ofrece recomendaciones de productos altamente especializadas a gran escala.
-

Angular

Angular es un framework para desarrollar aplicaciones SPA (Single Page Applications), de código abierto, mantenido por Google. Se basa en TypeScript, que es un superconjunto de Javascript. Typescript esencialmente agrega tipos estáticos y objetos basados en clases.

Angular se propone como objetivo, brindarle a las aplicaciones basadas en navegadores la posibilidad de pensarlas como aplicaciones en si misma, integrando patrones de diseño como MVC (Model View Controller) y mejorando los esfuerzos necesarios para la construcción y el desarrollo, además de permitir realizar pruebas unitarias que acompañen la construcción.

El framework lee el HTML que contiene atributos de las etiquetas personalizadas, entonces obedece a las directivas de los atributos personalizados, y une las piezas de entrada o salida de la página a un modelo representado por las

variables estándar de JavaScript. Usa clases tipo "Componentes", cuyas propiedades son las usadas para hacer el binding de los datos. En dichas clases tenemos propiedades (variables) y métodos (funciones a llamar).

Utiliza como soporte de compilación, ejecución y soporte de dependencias a [Node.js](#)

Comandos

npm run build: Genera el paquete que sirve para distribuir nuestra aplicacion

npm run start: Inicializa la aplicación en entorno local (<http://localhost:4200/>)

Archivos importantes

Package.json: En el se definen las referencias a los paquetes y dependencias que utilizara la aplicacion.

Tambien define una serie de scripts que pueden ser lanzadores rapidos de procesos que el desarrollador necesite correr durante la construccion.

tsconfig.json: Archivo de configuracion de TypeScript, en el se detallan las opciones basicas del compilador de TypeScript.

angular.json: Archivo de configuracion de Angular con referencias globales que luego las utiliza tanto en tiempo de compilacion como de ejecucion.

Index.html: Punto de entrada a la aplicacion.

main.ts: Cargador principal de la aplicación. Deja disponible todos los componentes y funcionalidades que nos brinda el framework de trabajo.

Este resumen se basa en los cuatro documentos PDF proporcionados, que cubren qué es Angular, diferentes arquitecturas web, la diferencia entre Módulos y Standalone Components, las herramientas necesarias para trabajar con Angular y la estructura de un proyecto básico.

Resumen de los PDFs sobre Angular

Este resumen se basa en los cuatro documentos PDF proporcionados, que cubren qué es Angular, diferentes arquitecturas web, la diferencia entre Módulos y Standalone Components, las herramientas necesarias para trabajar con Angular y la estructura de un proyecto básico.

1. ¿Qué es Angular y tipos de Aplicaciones Web? (Basado en "01 - Angular.pdf")

- **¿Qué es Angular?**
 - **Angular es un framework de desarrollo frontend.**
 - **Está basado en TypeScript.**
 - **Fue desarrollado por Google.**
 - Se utiliza para crear **aplicaciones web de una sola página (SPA - Single Page Applications)** y aplicaciones web dinámicas en general.
 - Angular es un framework, lo que significa que proporciona todo lo necesario para construir una aplicación sin necesidad de depender de otras librerías de terceros, aunque se pueden combinar.
 - Es **flexible entre versiones**, con sus puntos vitales manteniéndose compatibles con versiones futuras y pasadas a medida que se actualiza.
- **Tipos de Aplicaciones Web:**
 - **Single Page Application (SPA):**
 - **Cómo funciona:** Solo se carga una vez el HTML, CSS y JavaScript en el navegador. Luego, el contenido cambia dinámicamente **sin necesidad de recargar la página.**
 - **Ejemplo:** Gmail, Facebook, Trello.
 - **Proceso de Navegación en una SPA:** El usuario ingresa a la web, el navegador descarga una única página HTML y el JavaScript necesario. Cuando el usuario navega, **JavaScript se encarga de cambiar el contenido.** Solo se hacen peticiones al servidor para obtener datos (ej. API REST), pero no se vuelve a descargar toda la página.
 - **Ventajas de una SPA: Experiencia de usuario fluida y rápida. Menos carga en el servidor** después del primer acceso. **Ideal para aplicaciones web interactivas.**
 - **Desventajas de una SPA: SEO deficiente** porque los buscadores no ven el contenido inicial. **Tiempo de carga inicial más lento.** Puede consumir más recursos en el navegador.
 - **Server-side Rendering (SSR):**
 - **Cómo funciona:** El servidor renderiza la página completa en cada petición y la envía **lista para ser mostrada en el navegador.**

- **Ventajas de una SSR: SEO excelente. Carga inicial más rápida.** Funciona sin JavaScript en algunos casos. Buena experiencia en dispositivos lentos.
 - **Desventajas de una SSR: Carga más el servidor.** Las páginas no son instantáneas al navegar. Más difícil de escalar.
- **Static Site Generation (SSG):**
 - **Cómo funciona:** Las páginas se **generan una sola vez, en el momento de la compilación**, y luego se sirven como archivos estáticos. No necesita que el servidor genere contenido dinámico en cada petición.
 - **Ventajas de una SSG: Carga ultra rápida** porque el contenido ya está generado. **No hay carga en el servidor**, solo entrega archivos estáticos. **SEO excelente** porque los motores de búsqueda ven el HTML completo. **Escalabilidad alta** ya que puede servirse desde una CDN (como Vercel o Netlify).
 - **Desventajas de una SSG: No es útil para contenido que cambia constantemente.** Requiere reconstrucción del sitio si el contenido cambia. No permite personalización dinámica sin JavaScript adicional.
- **Partes Fundamentales de Angular:**
 - Los puntos fundamentales de Angular son 6. Hay más, pero estos son los más comunes.
 - **Componentes:** Son **piezas que representan una parte de la interfaz del usuario**. Pueden ser algo muy pequeño como un botón o toda una pantalla. Se dividen en: un archivo **.TS** (lógica), un archivo de estilo opcional (SASS, CSS), una plantilla HTML (interfaz), y un archivo **.spec.TS** (pruebas). Los archivos de estilos y HTML pueden estar dentro del TS, pero no se recomienda.
 - **Rutas:** Permiten **cambiar de páginas** (components que usualmente cubren todo el punto de vista). Sirven para separar lógica en pequeñas pantallas, control de acceso y autorización, y estrategias de renderización (lazy, llamados, etc.).
 - **Directivas: Modifican el comportamiento de un elemento HTML.** Ejemplos: Directivas de Atributo (**ngClass**, **NgStyle**), Directivas Estructurales (**ngIf**, **ngFor**), y Directivas de Componente (personalizadas).
 - **Pipes: Modifican cómo se presentan los datos.** Se usan para ordenar y filtrar, optimización de rendimiento, y son personalizables. Transforman datos de forma visual para representarlos apropiadamente en los componentes.
 - **Servicios:** Son **clases diseñadas para contener lógica reutilizable y gestionar datos compartidos entre componentes**.
 - **Módulos:** En Angular, un módulo es una forma de **organizar y estructurar la aplicación en partes reutilizables y mantenibles**. Se define mediante la clase **@NgModule** y **agrupa componentes, directivas, pipes y servicios relacionados**. Tipos de Módulos: Módulo raíz (**AppModule**), Módulos funcionales, Módulos compartidos (**SharedModule**), Módulos con Lazy Loading.

- **Mitos y Realidades de Angular:**

- Angular solo puede correr código TS en aplicaciones de Angular (esto es un mito o una generalización).
 - Angular es mejor que otros frameworks (realidad/opinión: es más ordenado).
 - Libera versiones a cada rato (realidad).
 - Es complicado de aprender (realidad o mito, depende del punto de vista).
 - Angular 2, 4, 5... 20... 22... es el mismo Angular (realidad, ha evolucionado pero mantiene la base).
-

2. Standalone Components vs Módulos (Basado en "02 - Standalone vs Modules.pdf")

Este documento compara dos enfoques para organizar la aplicación en Angular.

- **Módulos (@NgModule):**

- **Definición:** Una forma de organizar y estructurar la aplicación en partes reutilizables y mantenibles, agrupando componentes, directivas, pipes y servicios relacionados.
- **Ventajas:** Organizar la aplicación en módulos reutilizables. Facilitar la carga diferida (Lazy Loading) dividiendo la aplicación en partes más pequeñas. Controlar el alcance (scope) de componentes y servicios, evitando conflictos de dependencias. Facilitar la integración con bibliotecas externas que dependen de módulos. Separación clara de responsabilidades (feature modules, core module, shared module).
- **Ineficiencias:** Carga innecesaria de módulos en tiempo de ejecución. **Código repetitivo (Boilerplate)**, ya que cada módulo necesita **declarations, imports, providers y exports**. Más difícil de entender para nuevos desarrolladores en Angular. Puede generar sobrecarga en la administración de dependencias. Mayor cantidad de código y configuraciones innecesarias en proyectos pequeños.

- **Standalone Components:**

- **Definición:** El enfoque standalone permite definir componentes independientes **sin necesidad de agruparlos dentro de un módulo**. Cada componente puede **importar directamente sus propias dependencias**.
- **Ventajas:** **Ideal para microservicios, librerías de UI y pequeñas aplicaciones. Mejor rendimiento** al evitar la sobrecarga de módulos innecesarios. **Mejor integración con la carga perezosa (loadComponent())**. Permite una carga más rápida y **menos código boilerplate**. Simplifica la arquitectura eliminando módulos innecesarios. Facilita la adopción de Angular para nuevos desarrolladores.

- **Desventajas:** Puede volverse **difícil de escalar en proyectos grandes** sin una buena organización. **Cada Standalone Component debe importar manualmente todas sus dependencias.**
 - **Conclusión:** Angular está migrando hacia Standalone Components. En proyectos grandes, los NgModules aún son útiles. Si el proyecto lo permite, **optar por Standalone Components para una arquitectura más simple y ligera** es recomendable, y es probable que en futuras versiones los módulos sean menos necesarios.
-

3. Herramientas para la Instalación y Desarrollo (Basado en "03 - Instalaciones.pdf")

Este documento describe las herramientas esenciales para configurar un entorno de desarrollo Angular.

- **Node.js:**
 - Es un **entorno de ejecución de JavaScript** basado en el motor V8 de Google Chrome.
 - Permite ejecutar código JavaScript en el servidor.
 - Necesario porque Angular usa Angular CLI, que está escrita en Node.js.
 - Se recomienda la **última versión LTS (soporte a largo plazo)**.
- **NPM (Node Package Manager):**
 - Incluido con Node.js.
 - Permite **instalar y gestionar paquetes y librerías de terceros**.
 - Angular usa muchos paquetes que se instalan a través de npm.
- **Visual Studio Code (VS Code):**
 - **Editor de código fuente gratuito** desarrollado por Microsoft.
 - Es **ligero, rápido y altamente personalizable**.
 - **Características:** Soporte para múltiples lenguajes, Autocompletado inteligente (IntelliSense), Depuración integrada, Control de versiones con Git integrado, Extensiones y plugins, Terminal integrada, Multiplataforma (Windows, macOS, Linux), Gran comunidad y soporte, Integración con tecnologías modernas (Docker, Kubernetes, etc.).
- **Postman:**
 - Herramienta de desarrollo para **probar, desarrollar y documentar APIs**.
 - Permite enviar solicitudes HTTP y analizar respuestas sin escribir código manual en frontend/backend.
- **Git:**
 - **Sistema de control de versiones distribuido**.
 - Utilizado para **rastrear cambios en el código fuente**.

- Permite la colaboración eficiente de varios desarrolladores.
 - **Navegador:**
 - Se recomienda **Google Chrome**, ya que Google creó Angular.
 - **Instalación de Angular CLI:**
 - Se instala globalmente usando npm.
 - El comando es: `npm install -g @angular/cli` (ejecutar como administrador).
 - Angular CLI se usa para gestionar proyectos, compilar código y ejecutar pruebas.
 - **Extensiones Opcionales de VS Code:**
 - **Angular Language Service:** Ayuda a VS Code a entender la sintaxis de Angular.
 - **Angular Snippets:** Permite escribir código Angular más rápido.
 - **Angular Schematics:** Permite crear archivos de Angular más rápido.
 - **Auto Close Tag:** Cierre automático de etiquetas HTML.
 - **Auto import:** Importaciones automáticas.
 - **Error Lens:** Permite ver errores sin necesidad de pasar el cursor sobre el código.
 - **Paste JSON as Code:** Permite crear interfaces a partir de un objeto JSON.
 - **TypeScript Importer:** Ayuda con las importaciones de TypeScript (opcional).
 - **Editor Config for VSCode:** Trabaja con configuraciones por defecto de VS Code.
 - **Better Comments:** Permite ver comentarios con colores.
-

4. Estructura de un Proyecto Angular (Basado en "04 - Mi primera app.pdf")

Este documento explica la estructura de carpetas y archivos de un proyecto Angular generado.

- **Crear el primer proyecto de Angular:** Implica un proceso para iniciar un nuevo proyecto.
- **Archivos y Carpetas Clave:**
 - `.angular`: Maneja la cache para que la aplicación inicie más rápido en accesos posteriores.
 - `.vscode`: Contiene las configuraciones específicas de Visual Studio Code para el proyecto. Si se descarga el proyecto, el editor se ajustará según estas configuraciones.
 - `node_modules`: Contiene **todos los paquetes referenciados en el `package.json`**. No se debe modificar manualmente.

- **public**: Contiene **todos los recursos estáticos** de la aplicación, como imágenes, iconos, fuentes.
- **.editorconfig**: Contiene **configuraciones recomendadas por Angular** para el editor de código.
- **.gitignore**: Archivo para **ignorar archivos al subir a Git**.
- **.angular.json**: **Archivo de configuración de Angular**. Rara vez se toca, pero permite configurar estilos, js, modos de desarrollo y prueba.
- **package-lock.json**: **Archivo JSON generado automáticamente** en proyectos de Node.js. **No se toca manualmente**. Contiene las **versiones exactas de los paquetes y sus dependencias** instaladas en el proyecto. Se utiliza para construir los módulos de node.
- **package.json**: Contiene las **dependencias** del proyecto (tanto en modo desarrollo como en test). Define los **scripts para ejecutar** (levantar la app, modo test, etc.). Indica la **versión de la app** (Angular siempre actualiza todo a su misma versión).
- **README.md**: Contiene **explicaciones del proyecto**.
- **tsconfig.json**: **Archivo de configuración de TypeScript**. Define cómo se quiere que sea la versión de JS. No se recomienda tocarlo a menos que se sepa lo que se hace, ya que por defecto está como lo recomienda el equipo de Angular.
- **tsconfig.spec|app.json**: **Extienden de tsconfig.json**. Se usan uno cuando inicia la app (**tsconfig.app.json**) y otro cuando se testea (**tsconfig.spec.json**).
- **styles.css**: Contiene los **estilos globales** para todos los componentes.
- **index.html**: Es el **punto de entrada principal de una aplicación Angular en el navegador**. Es el archivo HTML base en el que Angular inyecta y renderiza la aplicación.
- **main.ts**: Es el **punto de entrada de una aplicación Angular**. Es el primer archivo que se ejecuta y su función principal es arrancar (**bootstrap**) el componente raíz (**AppComponent**).
- **app**: Carpeta donde se encuentran los **componentes** de la aplicación con sus archivos.
- **app.config.ts**: **Archivo de configuración central en Angular 19** que reemplaza la necesidad de usar **AppModule**. Define la configuración global utilizando **ApplicationConfig** en lugar del enfoque basado en **NgModule**. Un ejemplo de su uso es **provideRouter(routes)**, que define el enrutamiento.
- **Diferencia entre package.json y package-lock.json**:
 - **package.json** define las **dependencias del proyecto y sus versiones permitidas** usando rangos (como **^** y **~**).
 - **package-lock.json** registra las **versiones exactas y bloqueadas de todas** las dependencias y sub-dependencias instaladas, asegurando que las instalaciones sean consistentes en diferentes entornos.

- **Versionado Semántico (SemVer):**

- Es un **estándar** que define cómo se asignan los números de versión en el software.
- Sigue el formato: **MAJOR.MINOR.PATCH**.
 - **PATCH:** Corrección de errores o mejoras menores sin cambiar funcionalidades.
 - **MINOR:** Nuevas funcionalidades compatibles con versiones anteriores.
 - **MAJOR:** Cambios que **rompen compatibilidad** con versiones anteriores.

- **Símbolos `^` y `~` en `package.json`:**

- Controlan cómo se actualizan las dependencias al ejecutar `npm install`.
- `^` (caret): Permite actualizar versiones **minor y patch**, pero **mantiene la major**. Ejemplo: `"lodash": "^4.17.21"` permite versiones desde 4.17.21 hasta 4.x.x (pero no 5.x.x).
- `~` (tilde): Permite actualizar **solo patch**, pero **mantiene major y minor**. Ejemplo: `"lodash": "~4.17.21"` permite versiones desde 4.17.21 hasta 4.17.x (pero no 4.18.x).

Señales en Angular

Las Señales en Angular, implementadas a través de Zone.js por defecto, son un mecanismo que permite detectar cambios en las propiedades y notificar a la plantilla para que se vuelva a renderizar. Con **Signal**, la detección de cambios se realiza específicamente a nivel de la señal.

Una señal es un **contenedor que envuelve un valor y notifica a los consumidores interesados cuando este cambia**. Para interactuar con el valor de una señal, existen métodos principales:

- **Set:** Para establecer un nuevo valor.
- **Update:** Para actualizar el valor basándose en el valor actual.
- **Mutate:** Este método **fue eliminado en la versión 17 de Angular**.

Es importante entender cómo Angular (y otros frameworks reactivos) detectan cambios, lo cual se basa en la **referencia en memoria de objetos y arrays**. Si modificas un objeto o array asignando una nueva referencia, Angular detecta el cambio y reacciona.

Existen tipos especiales de señales:

- **Signal Computed:** Son señales de lectura cuyo **valor depende de otras señales**. Se actualizan automáticamente cuando cambia una señal de la que dependen, o al leerse su valor. Angular calcula su valor la primera vez que se lee y lo guarda en

memoria; si las señales dependientes no cambian, al leerse nuevamente, devuelve el valor ya calculado.

- **Effects**: Son operaciones que **se ejecutan cuando una o más señales dentro de su función cambian**. Por ejemplo, un efecto podría dispararse cada vez que cambian los productos. Es crucial **destruir los efectos** para evitar que queden en memoria.

Data Binding en Angular

El Data Binding es la **forma en que Angular conecta el componente TypeScript con la plantilla HTML**. Permite que los datos fluyan entre ellos en una o ambas direcciones. Hay **cuatro formas principales** de realizar Data Binding en Angular:

1. **Interpolación**: Permite **mostrar datos del componente a la plantilla HTML** insertando expresiones de TypeScript dentro de llaves dobles `{{ }}`. Se usa para mostrar valores de propiedades, evaluar expresiones o invocar métodos (aunque esto último no se recomienda abusar por rendimiento).
2. **Property Binding**: Es un enlace **unidireccional** (One-Way) que va **del componente al atributo HTML**. A diferencia de la interpolación, puede usarse con atributos DOM reales como `[value]`, `[disabled]`, `[class]`, etc..
3. **Event Binding**: Es un enlace **unidireccional** (One-Way) que va **del HTML al componente**. Permite reaccionar a eventos del usuario en la plantilla.
4. **Two-Way Data Binding**: Permite la **sincronización en ambas direcciones**. Los datos se actualizan automáticamente tanto si cambian en el componente como si cambian en la plantilla (por ejemplo, la entrada de un usuario en un input).

Rutas en Angular

Las rutas en Angular conforman el **sistema de navegación que carga componentes dinámicamente según la URL actual**. Se gestionan principalmente a través del `RouterModule` y la configuración en un archivo `app-routing.module.ts`. Para Standalone Components, la configuración se puede hacer directamente en `main.ts` u otro archivo de enrutamiento.

Al definir una ruta, se configuran propiedades como:

- `path`: Define la **URL**.
- `component`: El **componente que se renderizará** cuando se esté en la URL definida.
- `redirectTo`: Permite **redirigir** a otra URL si se ingresa a una ruta inválida.

Un concepto importante en rutas es el **Lazy Load**, que permite **cargar módulos solo cuando se necesitan**, mejorando el rendimiento. Esto es posible con módulos completos, pero no con componentes individuales mediante lazy load.

Para obtener información de la ruta activa, se usa el servicio **ActivatedRoute**. Permite acceder a parámetros y datos de la URL. Métodos para capturar información incluyen:

- **snapshot**: Toma una "foto" de los parámetros en el momento en que el componente se carga. Es útil para componentes que no necesitan reaccionar a cambios en la URL después de cargarse. Una limitación mencionada es que no es la forma más moderna y **paramMap** es mejor para evitar errores con **null**.
- **paramMap**: Es una forma más moderna para obtener parámetros de la URL, evitando errores con **null**. Es útil para componentes reutilizables que deben reaccionar a cambios en la URL.
- **queryParams**: Se usa para obtener parámetros de consulta en la URL (los que van después del **?**). Son útiles para filtros, búsquedas o paginación sin modificar la ruta principal. Usar **.get()** con **queryParams** es más seguro que acceder directamente con **['clave']** porque evita errores si el parámetro no está presente.

El servicio **Router** se utiliza para la **navegación programática**, permitiendo cambiar de ruta desde el código TypeScript.

Servicios e Inyección de Dependencia en Angular

Los servicios en Angular son **clases diseñadas para contener lógica reutilizable y gestionar datos compartidos** entre componentes. Ayudan a mantener el código modular y limpio al delegar tareas como peticiones HTTP, gestión de estados y lógica de negocio, liberando a los componentes de estas responsabilidades.

La **Inyección de Dependencias (DI)** es un patrón de diseño fundamental en Angular. Permite que las dependencias (los servicios que una clase necesita) le sean proporcionadas en lugar de que la propia clase las cree manualmente. En Angular, **no necesitas hacer `new MiServicio()`**, Angular se encarga de proporcionar la instancia automáticamente.

Las **ventajas de la DI** incluyen:

- **Reduce el acoplamiento**: Los componentes no dependen de la implementación exacta del servicio, facilitando cambios y pruebas.
- **Facilita el mantenimiento**: Si un servicio cambia, no es necesario actualizar cada lugar donde se utiliza.
- **Promueve la reutilización**: Un servicio puede ser usado en múltiples componentes sin crear múltiples instancias.
- **Facilita las pruebas unitarias**: Permite "mockear" servicios para probar componentes aisladamente.
- **Evita la creación manual de instancias**.

La diferencia clave entre usar **`new ServiceX()`** e inyectar un servicio es que cada **`new`** crea una nueva instancia distinta, dificultando compartir datos. Al inyectar un servicio,

especialmente cuando se define con `@Injectable({ providedIn: 'root' })`, Angular lo gestiona como un **Singleton** (una única instancia para toda la aplicación) dentro de su ámbito, lo que facilita la gestión de estado y el uso compartido de datos. Los servicios también pueden usarse bajo otros patrones como Strategy, Factory u Observer.

Las **responsabilidades comunes de un servicio** incluyen:

- **Peticiones HTTP:** Encapsular llamadas a APIs externas. Esto mantiene los componentes limpios, facilita la reutilización y simplifica el testing. Un **mal enfoque** sería que el componente hiciera la petición directamente.
- **Gestión de estados:** Actuar como un almacén de datos compartido para mantener un estado consistente entre componentes. Un **mal enfoque** sería guardar el estado localmente en el componente, lo que causa problemas si el componente se destruye o si otros componentes necesitan esos datos. Un **buen enfoque** es usar un servicio de estado.
- **Lógica de negocios:** Manejar validaciones, transformaciones y cálculos. Un **mal enfoque** sería que un componente tuviera demasiada lógica. Un **buen enfoque** es mover esta lógica a un servicio.
- **Seguridad y autenticación:** Gestionar tokens y permisos.
- **Compartir datos entre componentes:** Evitando múltiples capas de comunicación directa.
- **Implementación de patrones de diseño.**

HttpClient en Angular

HttpClient es un servicio de Angular, parte del paquete `@angular/common/http`, que se utiliza para **realizar peticiones HTTP** para comunicarse con servidores y APIs. Proporciona métodos para las operaciones CRUD (GET, POST, PUT, DELETE, PATCH, etc.).

HttpClient es más eficiente que otras opciones como XMLHttpRequest o fetch debido a sus características:

- **Soporta Observables:** Permite un **manejo reactivo** con RxJS.
- **Interceptores:** Permiten **modificar peticiones y respuestas** globalmente.
- **Manejo de errores:** Con operadores como `catchError()`.
- Configuración de Headers, Parámetros y Autenticación.

Para usar HttpClient en Standalone Components, se configura en `app.config.ts`. Permite enviar **Headers** (para autenticación o personalización) y **Parámetros** (para filtrar datos sin modificar la URL manualmente). El manejo de errores con `catchError` es fundamental para **capturar y manejar errores sin romper la aplicación**.

Los **HttpInterceptors** son clases especiales que **interceptan todas las peticiones y respuestas HTTP**. Se usan para modificar, analizar o manejar errores antes de que lleguen al componente. Son útiles para:

- Agregar **autenticación** (tokens, JWT).

- **Manejar errores de forma centralizada.**
- Agregar **logs**.
- **Transformar datos.**
- **Cachear respuestas.**

En conclusión, HttpClient es una **herramienta poderosa** que facilita las peticiones HTTP con características avanzadas como Interceptores, manejo centralizado de errores y configuración flexible.

Observables y RxJS en Angular

Los **Observables en Angular** son una implementación de la librería **RxJS** (Reactive Extensions for JavaScript). Permiten **manejar flujos de datos y eventos**. Pueden ejecutar su lógica de manera **sincrónica o asíncrona**. Angular los usa principalmente para peticiones HTTP con HttpClient, eventos y formularios reactivos, interacción con el DOM, y streams de datos en tiempo real.

Las **ventajas de los Observables** incluyen:

- **Manejo de datos asíncronos:** Permiten gestionar flujos que ocurren en el futuro sin bloquear la ejecución del código.
- **Operadores de RxJS:** RxJS proporciona operadores para **transformar, combinar y filtrar datos**.
- **Cancelación de peticiones:** Permiten **cancelar peticiones en curso**, evitando fugas de memoria y mejorando el rendimiento. Por ejemplo, **switchMap** es útil para cancelar peticiones anteriores cuando se inicia una nueva (como al cambiar de chat).
- **Menor acoplamiento:** La lógica de obtención de datos puede estar separada del componente que la usa, típicamente en un servicio.
- **Flexibilidad:** Permiten trabajar con cualquier tipo de flujo de datos (HTTP, eventos, intervalos, etc.) de manera unificada.
- **Composición de flujos:** Permiten combinar varios flujos de datos para simplificar la lógica.

Aunque los Observables pueden ser asíncronos (como una petición HTTP que espera respuesta), también pueden ser **sincrónicos**, emitiendo sus valores inmediatamente en el mismo ciclo de ejecución. Un Observable sincrónico es inmediato y bloqueante hasta que termina, sin retrasos ni procesos en segundo plano.

Las **desventajas de los Observables** son:

- **Curva de aprendizaje pronunciada:** Requiere entender bien los operadores de RxJS.
- **Mayor complejidad en comparación con Promises:** Un Promise es más sencillo para una única respuesta asíncrona. Los Observables manejan múltiples valores en el tiempo, requieren suscripción (**subscribe()**) para empezar a emitir datos (a

diferencia de un Promise que ejecuta su lógica de inmediato), y deben gestionarse manualmente.

- **Posibles fugas de memoria:** Si no se gestionan bien las suscripciones (por ejemplo, olvidando `unsubscribe()`), los Observables pueden seguir vivos y consumir recursos después de que el componente se destruya. Soluciones como `takeUntil` ayudan a evitar estas fugas.

En resumen, los Observables son **fundamentales en Angular para el manejo de datos asíncronos y reactivos**, potenciados por RxJS. Son más flexibles que las Promises pero requieren una curva de aprendizaje y una gestión cuidadosa de las suscripciones.

Pipes en Angular

Un **Pipe** (o "tubería") es una función especial en Angular que **transforma datos directamente en el template (HTML)**, sin necesidad de modificar el valor original en el componente.

Sirven para:

- **Formatear datos** rápidamente en la vista.
- Mejorar la **legibilidad** del código.
- Evitar lógica innecesaria en el componente o el template.

Son útiles cuando necesitas transformar valores directamente en el HTML, mostrar fechas, números o textos con formatos específicos, o si repites una transformación (puedes crear un pipe personalizado).

Las fuentes mencionan varios **pipes integrados**:

- **Lowercase Pipe:** Transforma un string a minúsculas.
- **Uppercase Pipe:** Transforma un string a mayúsculas.
- **Titlecase Pipe:** Pone en mayúscula la primera letra de cada palabra.
- **Date Pipe:** Edita una fecha para mostrarla en un formato especificado.
- **Decimal Pipe (o Number Pipe):** Formatea un valor numérico según opciones de dígitos y configuración regional. Permite controlar la cantidad mínima de enteros (rellenando con ceros) y el rango de decimales.
- **Currency Pipe:** Muestra un número en formato moneda, permitiendo especificar la moneda, cómo mostrar el símbolo y el formato de los dígitos.
- **Percent Pipe:** Muestra un número como porcentaje, permitiendo controlar el formato de los dígitos.
- **i18nSelectPipe:** Muestra una cadena que coincide con un valor actual.
- **i18nPluralPipe:** Asigna un valor a una cadena que pluraliza el valor según reglas de configuración regional.
- **Json Pipe:** Convierte un valor a su representación en formato JSON, útil para depuración.
- **KeyValue Pipe:** Transforma un objeto en un array de pares clave-valor (key-value).
- **Slice Pipe:** Mencionado en la lista.

Además de los pipes integrados, Angular permite crear **Pipes personalizados**. Se generan usando el comando `ng generate pipe nombre` o `ng g p nombre`.

NODE

Node.js es un entorno de tiempo de ejecución de JavaScript de código abierto y multiplataforma. Ejecuta el motor JavaScript V8, el núcleo de Google Chrome, fuera del navegador, permite que Node.js sea muy eficaz

Se ejecuta en un solo proceso, sin crear un nuevo hilo para cada solicitud. Proporciona un conjunto de primitivas de E/S asíncronas en su biblioteca estándar que evita que el código JavaScript se bloquee y, en general, las bibliotecas en Node.js se escriben utilizando paradigmas que no bloquean, lo que hace que el comportamiento de bloqueo sea la excepción en lugar de la norma.

Cuando realiza una operación de E/S, en lugar de bloquear el subproceso y desperdiciar ciclos en espera, reanudará las operaciones cuando regrese la respuesta.

Primero incluye el módulo http de Node.js, para soporte de operaciones con redes. El método `createServer()` de http crea un nuevo servidor HTTP y lo devuelve.

El servidor está configurado para escuchar en el puerto y el nombre de host especificados en el método `listen`. Cuando el servidor está listo, se llama a la función de callback, en este caso nos informa que el servidor se está ejecutando.

Cada vez que se recibe un request, se llama a la función dentro del `createServer`, proporcionando dos objetos: una request y un response

nodejs: Callbacks

Funciones que en general se pasan como parámetro a otras. Tienen como finalidad tener referencia en el código llamador de cuando ocurre el evento de respuesta ante una llamada.

En esta función se puede incluir cualquier tipo de procesamiento posterior a la invocación en curso que se lanzo.

En general son utilizadas en operaciones asincrónicas, y en lugar de bloquear el código principal o llamador a la espera de que finalice la funcionalidad invocada, esta informa de su finalización a través de la función callback.

nodejs: Módulos

Node permite la incorporación de módulos o librerías propias o de terceros.

Para librerías de terceros, posterior a instalar la librería con: **npm install "nombreLibreria"**

Declaramos la necesidad de utilizarla

Ejemplo:

```
const referenciaLibreria = require('nombreLibreria');
```

Para librerías propias, primero debemos crear un archivo con la declaración y la exportación de lo que queremos importar o utilizar. Luego se declara la referencia para poder utilizarla

```
const miModulo = require('./mimodulo');
```

nodejs: Herramientas de terceros

Lodash: Biblioteca moderna ofrece modularidad, rendimiento, trabajo con matrices, números y objetos, etc. (<https://lodash.com/>)

yargs: Ayuda a crear herramientas de comando interactivas, analizando argumentos. (<http://yargs.js.org/>)

Herramientas propias de node

FileSystem: El fs Proporciona una gran cantidad de funciones muy útiles para acceder e interactuar con el sistema de archivos.
(<https://nodejs.dev/learn/the-nodejs-fs-module>)

No hay necesidad de instalarlo. Al ser parte del núcleo de Node.js, se puede usar simplemente solicitándolo:

```
const fs = require ( 'fs' ) ;
```

Una vez que lo haga, tendrá acceso a todos sus métodos, algunos ejemplos:

fs.copyFile(): copia un archivo

fs.mkdir(): crear una nueva carpeta

fs.readFile(): leer el contenido de un archivo. Relacionada:fs.read()

fs.rename(): cambiar el nombre de un archivo o carpeta

fs.rmdir(): eliminar una carpeta

fs.writeFile(): escribir datos en un archivo. Relacionada:fs.write():

os: Proporciona muchas funciones que puede usar para recuperar **información del sistema operativo** subyacente y la computadora en la que se ejecuta el programa, e interactuar con él.

os.arch(): Devuelve la cadena que identifica la arquitectura subyacente, como arm, x64, arm64.

os.cpus(): Devuelve información sobre las CPU disponibles en su sistema.

os.freemem(): Devuelve el número de bytes que representan la memoria libre en el sistema.

os.homedir(): Devuelve la ruta al directorio de inicio del usuario actual.

nodejs: Promise

Una promesa es una forma avanzada de trabajar con operaciones asíncronas. En lugar de utilizar una función a modo de callback utilizamos una estructura.

Donde el llamador puede manipular la respuesta con la instrucción then. Dentro de esta puede diferenciar claramente, ejecuciones con error de ejecuciones correctas.

La función asíncronica invocada que desea retornar una promesa, contara con las funciones res y rej, para indicar que finalizó correctamente o con errores respectivamente.

Modulos utiles

Lodash: Simplifica el manejo y edición de objetos, estructuras y arrays.

<https://www.npmjs.com/package/lodash>

```
const _ = require('lodash');
```

```
let x = { "nombre": "Jaime" }
let y = { "apodo": "pepito" }
let z = [
  { nombre: "Jaime", apellido: "Medina", edad: 26 },
  { nombre: "Jose", apellido: "Perez", edad: 31 }
]

_.times(3, ()=> console.log('Ejecutar varias veces'));

let resultado = _.find(z, {nombre: "Jaime", apellido: "Medina"});
```

yargs: Permite manipular argumentos de una manera intuitiva.

<https://www.npmjs.com/package/yargs>

```
const argv = require('yargs').argv;

if (argv.usuario === 'prueba') {
```

File System: Permite trabajar con el Sistema de archivos.

```
3-file_system > JS app.js > ...
1  const fs = require('fs');
2
3  console.log('Iniciado')
4
5  fs.readdir('./../3-file_system', (error, files)=>{
6    files.forEach(file => {
7      console.log(file);
8    });
9  });
10
11 console.log('Finalizado')
12
```

Promesas: Estructuras que representan la correcta finalización o la finalización con errores de una operación asíncrona.

```
JS app.js  JS promesa.js X
4-promise > JS promesa.js > calcular > <function>
1  let calcular = (numero1, numero2) => {
2    return new Promise((res, rej) => {
3      setTimeout(() => {
4        let suma = numero1 + numero2;
5        if(suma > 5){
6          res(numero1 + numero2);
7        }else{
8          rej('Error al procesar los datos')
9        }
10     }, 2000);
11   });
12 }
13
14
15 module.exports = {
16   calcular
17 }
```

```
JS app.js x JS promesa.js
4-promise > JS app.js > ...
1  const prom = require('./promesa');
2
3  prom.calcular([5,2]).then((resultado) => {
4    console.log(resultado);
5  }, (error)=>{
6    console.log(error);
7  });
8
9  console.log('Fin de codigo principal')
10
```

JSON Manipulacion de objetos con estructura JSON.

```
33
34  JSON.stringify
(method) JSON.stringify(value: any, replacer?: (this: any, key: string, value: any) => any, space?: string | number): string (+1 overload)

Converts a JavaScript value to a JavaScript Object Notation (JSON) string.

@param value — A JavaScript value, usually an object or array, to be converted.

@param replacer — A function that transforms the results.

@param space — Adds indentation, white space, and line break characters to the return-value JSON text to make it easier to read.
```

```
5-http > JS geocode.js > ...
28
29
30
31
32
33
34  JSON.parse()
parse(text: string, reviver?: (this: any, key: string, value: any) => any): any

A valid JSON string.

Converts a JavaScript Object Notation (JSON) string into an object.
```

Servicios Rest Conjunto de especificaciones para servicios HTTP, que permiten construir servicios ligeros, claramente definidos y adaptables a cualquier tipo de tecnologia y cualquier tipo de consumidores.

Servicios Restfull Clientes - Servidor

- Sin estado
- Cacheable
- Interfaz uniforme
- Sistema de capas escalable

Metodos

- **GET** obtener recurso.
 - **POST** crear un nuevo recurso
 - **PUT** Editar un recurso totalmente
 - **PATCH** Realiza actualizaciones parciales
 - **DELETE** Eliminar un recurso.
-
- **HEAD** Test si existe el recurso antes de hacer la petición GET
 - **OPTIONS** permite al cliente conocer las opciones o requerimientos asociados a un recurso antes de iniciar cualquier petición sobre el mismo